

Universidad Nacional Autónoma de México

Facultad de Ciencias

Cómputo Concurrente

2027-1

Tarea 2. Problemas Clásicos

ALUMNOs:

Martínez Hidalgo Paola Mildred - 319300217
Dominguez Sanchez Gustavo Alonso - 418047121

Fecha de entrega:
16 de septiembre de 2026

Profesor:
Gilde Valeria Rodríguez



Ejercicios

1. **[15 min/15 puntos]** Muestra un ejemplo de una ejecución de la implementación de *Filter* en la que a un hilo A lo rebasa más de una vez cualquier hilo B. Considera un número n de hilos mayor a 10. Explica por qué no se cumple la propiedad de Justicia.

   Hint: Puedes mostrar una imagen de una ejecución o describirla.

   En este caso tomaremos $n = 11$ hilos $(T_0, T_1, T_2, \ldots, T_{10})$. De estos hilos consideraremos al hilo A (T_0) y al hilo B (T_1) como base, donde el hilo B será el que rebase más de una vez al hilo A.

   Recordando el algoritmo Filter, utilizamos dos arreglos globales:

   - `level[i]`: indica el nivel actual del hilo i (donde 0 significa fuera de la sección crítica).
   - `victim[L]`: indica qué hilo fue el último en llegar al nivel L y, por lo tanto, cede el paso en caso de conflicto.

   Cada hilo T_i ejecuta el siguiente algoritmo para entrar a la Sección Crítica (SC):

   ```
   for (L = 1; L < n; L++) {
       level[i] = L;
       victim[L] = i;
       while ((exists k != i such that level[k] >= L) && victim[L] == i) {
           // espera activa
       }
   }
   ```

   Y al salir de la Sección Crítica ejecuta:

   $$level[i] = 0$$

   Estado inicial

   - El hilo A (T_0) inicia su protocolo de entrada al Nivel 1. Ejecuta **level[A] = 1** y **victim[1] = A**.
   - El hilo A queda atorado en el Nivel 1 esperando en el `while` debido a que **victim[1] == A** y hay otros hilos activos.

   Primer rebase de B

   - El hilo B (T_1) inicia su ejecución e intenta entrar al Nivel 1. Ejecuta **level[B] = 1** y escribe **victim[1] = B**.
   - Al convertirse **B** en la nueva víctima del Nivel 1 (`victim[1] == B`), el hilo A se libera de su condición de espera.
   - A avanza inmediatamente al Nivel 2, ejecuta **level[A] = 2** y **victim[2] = A**. Ahora A queda atorado en el Nivel 2.

- Mientras tanto, los hilos $T_2, T_3, \ldots, T_{10}$ avanzan rápidamente tomando el rol de víctima sucesivamente en niveles superiores.
- El hilo B (T_1) aprovecha este movimiento, sube rápidamente por todos los niveles (L_1 a L_{10}) desplazando a otros hilos, entra a la Sección Crítica (SC) y sale (**level[B] = 0**).

Segundo rebase de B

- El hilo A permanece congelado en el Nivel 2 (`victim[2] == A`).
- El hilo B vuelve a iniciar su protocolo de entrada desde cero. B entra al Nivel 1 y luego al Nivel 2 escribiendo **victim[2] = B**.
- Al ser B la nueva víctima del Nivel 2, desbloquea a A. El hilo A avanza al Nivel 3 y queda atorado como **victim[3] = A**.
- B se mueve ágilmente por los niveles superiores, supera nuevamente a A, entra por segunda vez a la Sección Crítica y sale.

Este patrón se puede repetir indefinidamente: B puede entrar N veces a la Sección Crítica mientras A es arrastrado lentamente nivel por nivel.

¿Por qué no se cumple la propiedad de Justicia?

El algoritmo Filter no utiliza estampas de tiempo (*timestamps*) ni colas FIFO para registrar el orden de llegada de las peticiones; únicamente mantiene el estado instantáneo actual (`level` y `victim`). Dado que no existe una garantía de First-Come, First-Served (FCFS), un hilo más rápido puede rebasar un número ilimitado de veces a un hilo más lento.

2. [30 min/25 puntos] Argumenta si la siguiente implementación (*VictimAround*) de un candado para dos hilos cumple con:

a) Exclusión mutua.

En este caso sí se cumple con la propiedad de exclusión mutua, en cuanto a la invariante tenemos que si un hilo i está dentro de la sección crítica, entonces `victim` = j (el nombre del otro hilo), e i no vuelve a escribir en `victim` mientras siga adentro (solo lo hará hasta que llame a `unlock()` y vuelva a intentar entrar).

Justificación: Para que un hilo i salga del ciclo externo y entre a la sección crítica, necesita leer `victim` ≠ `me`. Como solo hay dos hilos, eso significa que `victim` debe valer exactamente j. Esto solo puede pasar si la *última* escritura sobre `victim` fue hecha por j.

Una vez que i ya está adentro, no vuelve a tocar `victim` hasta terminar y hacer `unlock()`. Si mientras tanto j intenta entrar, se queda atorado en el ciclo interno porque `busy` ya vale `true`.

Supongamos, por contradicción, que los dos hilos (0 y 1) están adentro al mismo tiempo, mientras 0 está adentro, `victim = 1`; y mientras 1 está adentro, `victim = 0`. Pero `victim` es una sola variable: no puede valer 0 y 1 a la vez. Dado lo anterior tenemos una contradicción y por lo tanto no podemos tener ambos hilos dentro, esto cumple con exclusión mutua.

b) Deadlock-free.

En este caso no se cumple, para esto podemos poner como ejemplo qué pasaría si sólo tenemos un hilo. Supongamos que tenemos el hilo A, este es el único hilo que llama a `lock()`.

t_1: A ejecuta la línea 7: `victim = A`.

t_2: A evalúa la línea 8: `busy == false` (nadie más ha tocado `busy`) $\Rightarrow$ sale del ciclo interno.

t_3: A ejecuta la línea 9: `busy = true`.

t_4: A evalúa la línea 10: `victim == A`? $\Rightarrow$ **verdadero** (nadie más escribió `victim`) $\Rightarrow$ el ciclo externo se repite.

t_5: A vuelve a la línea 7: `victim = A` (otra vez, sigue siendo su propio nombre).

t_6: A evalúa la línea 8: `busy == true` (él mismo lo puso en t_3, y nadie hará `unlock()` porque A nunca llegó a la sección crítica) $\Rightarrow$ A queda atrapado repitiendo `victim = A` para siempre.

En pocas palabras A nunca regresa de `lock()`, y ningún otro hilo completa pasos ya que sólo tenemos uno. Esto viola la definición de *deadlock-free*, ya que el algoritmo necesita *competencia* para poder avanzar. Como tenemos un solo hilo siempre es el último en escribir la nota, así que siempre se ve a sí mismo al revisarla, y por lo tanto nunca lo dejan pasar.

c) Starvation-free.

Como Deadlock-free no se cumple, podemos saber que Starvation-free tampoco se va a cumplir, para esto ahora definamos dos hilos A y B los cuales compiten por adquirir el candado:

1) Ambos hilos observan que `busy == false`.
2) El hilo A ejecuta `victim = A`.
3) El hilo B ejecuta `victim = B`.
4) Ambos hilos establecen `busy = true`.
5) El hilo A observa que `victim != A` y entra en la sección crítica.
6) El hilo B observa que `victim == B` y debe volver a intentarlo.
7) El hilo A libera el candado.
8) Se repiten los pasos anteriores, favoreciendo nuevamente al hilo A.

Esta ejecución puede repetirse indefinidamente, de modo que el hilo B nunca consigue entrar en la sección crítica y por lo tanto no se cumple que cada hilo pueda avanzar eventualmente.

Hint: Si no lo cumple, describe un ejemplo de una ejecución. Si lo cumple, encuentra una invariante y justifica el porqué.

```
class VictimAround implements Lock {
    Shared Variables:
        1: int victim;
        2: boolean busy = false;
        3: void lock() {
        4:     int me = ThreadID.get();
        5:     do {
        6:         do {
        7:             victim = me;
        8:         } while(busy == true);
        9:         busy = true;
        10:    } while(victim == me);
        11: }
        12: void unlock() {
        13:     busy = false;
        14: }
}
```

Figura 1: Implementación de un candado "VictimAround".

3. **[40 min/25 puntos]** El problema clásico de los prisioneros y el interruptor.

   Eres uno de los n prisioneros arrestados. El alcaide de la prisión, un excomputólogo, reúne a todos los prisioneros en el patio y les anuncia el siguiente reto:

   > "A partir de mañana, serán confinados en celdas individuales de aislamiento sin ninguna posibilidad de comunicarse entre ustedes. Sin embargo, hoy tienen una hora para acordar una estrategia."

   Las reglas del reto son las siguientes:

   - Hay una habitación especial que contiene un único interruptor.
   - Se sabe con certeza que, al iniciar el reto, el estado inicial del interruptor es OFF (apagado).
   - El interruptor no está conectado a nada externo; su única función es mantener su estado (ON u OFF).
   - Cada día, el alcaide elegirá a un prisionero de manera aleatoria y lo llevará a la habitación del interruptor.
   - El alcaide garantiza justicia (*fairness*) en su elección: si el juego dura lo suficiente, cada prisionero visitará la habitación infinitas veces.
   - El prisionero que entra a la habitación puede observar el estado del interruptor y, si lo desea, cambiarlo (de ON a OFF, o de OFF a ON), o dejarlo como está.
   - En cualquier momento, si el prisionero en turno está absolutamente seguro, puede anunciar al alcaide: "Todos los n prisioneros hemos visitado esta habitación al menos una vez".

- Si tiene razón, todos son liberados. Si se equivoca, permanecerán encerrados para siempre.

Instrucciones:

a) Diseña un algoritmo concurrente (estrategia) que garantice que los prisioneros sean liberados. Argumenta paso a paso por qué tu solución es correcta y segura frente a cualquier orden de ejecución del alcaide.

Hint: En algoritmos concurrentes, a veces es útil romper la simetría teniendo un proceso que actúe como "coordinador" o contador, mientras los demás actúan como procesos regulares.

La estrategia que tendrán los prisioneros es que antes del aislamiento los n prisioneros se pondrán de acuerdo y harán lo siguiente:

- Lo primero que se hará será contar el total de prisioneros que hay y designar a **UN ÚNICO** prisionero como el **contador**. Para designar a este contador ya dependerá de cómo lo quieran hacer, esto puede ser con el prisionero que lleve más tiempo, el que lleve menos tiempo, el que tenga el número de identificación más alto o bajo, aquí ya dependerá de cómo se organicen los prisioneros.
- Los prisioneros restantes serán tomados como un **prisionero común.**
- Los prisioneros que son tomados como un **prisionero común** van a tener una variable privada, en otra palabras ellos irán informando por medio de una clave cuando ingresen por primera vez a la habitación.

$$\text{avisó}_p \in \{\text{falso}, \text{verdadero}\}, \quad \text{inicializada en falso.}$$

- El **contador** mantiene una variable local privada *cuenta* $\in \mathbb{N}$, inicializada en 0, en otras palabras el contador como lo dice su nombre irá contando las veces que un prisionero entró a la habitación.

Protocolo de un prisionero común p (en cada visita)

```
si (interruptor == OFF) y (aviso_p == falso):
    interruptor <- ON
    aviso_p <- verdadero
si no:
    no hacer nada
```

Interpretación:

Cuando un prisionero Normal p entra a la habitación y encuentra el interruptor en OFF, esto significa que **no hay ningún aviso pendiente sin consumir** en este momento. Esto puede deberse a dos razones distintas:

- Todavía nadie ha avisado, **o**

- Alguien ya avisó antes, pero el Contador ya pasó por la habitación y lo apagó (es decir, ya "consumió" ese aviso).

Sin embargo, encontrar el interruptor en OFF **no es suficiente** para que p decida encenderlo. Es necesaria una **segunda condición**: que p **nunca haya avisado antes** (avisó$_p$ = falso). Esto es importante porque si p ya avisó una vez (aunque el interruptor esté de nuevo en OFF más adelante), **no debe volver a encenderlo**, porque de lo contrario el contador lo contaría dos veces y creería, erróneamente, que hay más prisioneros distintos de los que realmente han visitado la habitación. Por eso la condición usa el conector **"y"** porque deben cumplirse ambas cosas, no solo la primera.

Solo cuando **ambas** condiciones se cumplen (interruptor = OFF **y** avisó$_p$ = falso), p enciende el interruptor y marca avisó$_p$ ← verdadero para nunca volver a hacerlo.

Si, en cambio, p encuentra el interruptor en ON, no debe hacer nada: deja que sea el **Contador** quien lo observe, lo apague y sume 1 a su cuenta. Esto asegura que cada encendido sea contado **exactamente una vez**, y por el prisionero correcto que lo generó.

Protocolo del Contador (en cada visita)

```
si (interruptor == ON):
    interruptor <- OFF
    cuenta <- cuenta + 1

si (cuenta == n - 1):
    anunciar "los n prisioneros ya visitaron la habitacion"
```

Interpretación:

Cuando el **Contador** entra a la habitación y encuentra el interruptor en ON, esto significa que hay exactamente **un aviso pendiente** dejado por algún prisionero común que aún no había sido contado. El Contador hace dos cosas:

- **Apaga el interruptor** (interruptor ← OFF): esto "libera el casillero" para que otro prisionero común (uno que todavía no haya avisado) pueda encenderlo más adelante. Este paso es indispensable: si el Contador no apagara el interruptor, ningún otro prisionero podría volver a avisar (porque su condición exige encontrarlo en OFF).
- **Incrementa su cuenta** ($cuenta \leftarrow cuenta+1$): registra que hay un prisionero común más que ya visitó la habitación. Como cada prisionero (menos el contador) enciende el interruptor a lo sumo una vez en toda la partida, este incremento corresponde, sin ambigüedad, a un prisionero **distinto** de todos los contados anteriormente.

Si en cambio el Contador encuentra el interruptor en OFF, no hace nada en esta parte: significa que no hay ningún aviso nuevo por consumir en este momento.

Después de esto (en la misma visita), el Contador revisa si ya alcanzó el total esperado: $cuenta = n - 1$. Este valor corresponde exactamente al número de prisioneros **Comunes** (todos excepto el propio Contador). Si la igualdad se cumple, el Contador puede anunciar

con **total certeza** que los n prisioneros ya visitaron la habitación al menos una vez.

Este algoritmo es seguro y confiable porque cada prisionero común tiene la orden de avisar una única vez, por lo que el contador tendrá la seguridad de que ya pasaron TODOS y que no está contando a un prisionero dos o más veces.

4. **[15 min/15 puntos]** Justicia y el Algoritmo de Peterson.

Considera la implementación del algoritmo de Peterson para 2 hilos:

```
class Peterson implements Lock {
    private boolean[] flag = new boolean[2];
    private volatile int victim;

    public void lock() {
        int i = ThreadID.get();
        int j = 1 - i;
        flag[i] = true;       // 1. Me interesa
        victim = i;           // 2. Cedo el paso (cortesa)

        // Espero si:
        // El otro quiere pasar Y yo soy la victima
        while (flag[j] && victim == i) {}
    }

    public void unlock() {
        flag[ThreadID.get()] = false;
    }
}
```

Recuerda la definición formal de Justicia (*First-Come-First-Served*/FIFO) vista en clase.

Dividimos el método `lock()` en dos secciones:

- **Doorway (Entrada):** Intervalo finito de pasos (código sin bucles infinitos).
- **Waiting (Espera):** El bucle `while` donde se puede esperar indefinidamente.

La propiedad FIFO establece que si el Hilo A termina su *Doorway* antes de que el Hilo B empiece el suyo ($D_A \rightarrow D_B$), entonces A debe entrar a la Sección Crítica antes que B ($CS_A \rightarrow CS_B$).

Responde lo siguiente:

a) Identifica y escribe exactamente qué líneas de código corresponden a la sección *Doorway* y cuáles a la sección *Waiting* en el algoritmo de Peterson.
Basándonos en la definición (Doorway que son los pasos finitos antes de esperar, y Waiting es el ciclo donde te puedes quedar atascado), las secciones en el código de Peterson son:
- Doorway (Entrada): Aunque las declaraciones de variables locales `int i` e `int j`, se podría considerar que también son parte del doorway (ya que son pasos finitos

antes del waiting), las escrituras en memoria compartida `flag` y `victim` son las que importan.

```
flag[i] = true; // 1. Me interesa
victim = i;     // 2. Cedo el paso
```

- Waiting (Espera): Es claro que ciclo `while` es el indicador principal para esta sección. Es posible que pueda esperar indefinidamente.

```
// Espero si: El otro quiere pasar y yo soy la victima
while(flag[j] && victim == i){}
```

b) Demuestra analíticamente si el algoritmo de Peterson cumple o no cumple con esta propiedad de Justicia (FIFO). Para ello, analiza el escenario donde el Hilo A termina su *Doorway* y se pausa, y justo después el Hilo B empieza y ejecuta su *Doorway* completo. ¿Quién logra pasar la sección de *Waiting* primero y por qué?
Dados los hilos A y B, si el doorway de A (D_1) termina antes que que el de B (D_B), recordemos que se cumple la propiedad FIFO si A entra a la sección crítica (CS_A) antes que lo haga B (CS_B):

$$D_A \rightarrow D_B \quad \Longrightarrow \quad CS_A \rightarrow CS_B \tag{1}$$

De acuerdo al escenario planteado, analicemos el valor de las variables en cada paso:

1) El Hilo A ejecuta su Doorway y luego se pausa:
   - A ejecuta `flag[A] = true`.
   - A `ejecuta victim = A`.
   - La memoria queda al momento como:

Variable	Valor
`flag[A]`	`true`
`flag[B]`	`false`
`victim`	`A`

   - El Hilo A se pausa.
2) El Hilo B inicia y ejecuta su Doorway:
   - B ejecuta `flag[B] = true.`
   - B ejecuta `victim = B`.
   - La memoria queda al momento como:

Variable	Valor
`flag[A]`	`true`
`flag[B]`	`true`
`victim`	`B`

3) En este punto tenemos dos opciones en Waiting:
   - **El hilo A queda pausado**: El hilo B seguira evaluando su bucle `while(flag[B] victim == A)`, donde ambas condiciones son verdaderas (debido a que B fue el último en escribirse a si mismo como `victim`). Por lo tanto B entra al ciclo y se queda esperando indefinidamente, sin entrar a la sección crítica.

- **El hilo A se reanuda**: A evalua su bucle `while(flag[B] victim == A)`, donde la última condición es falsa (debido a que B fue el último en escribirse a si mismo como `victim`). Por lo tanto, A no entra al ciclo y entra la sección crítica (CS_A). Eventualmente, si A termina sin errores su CS_A, entonces B rompería su ciclo y ejecutaría la sección crítica (CS_B).

Al revisar ambos casos, se cumple que si el hilo A, al haber llegado y terminado primero su doorway, logra pasar a la sección de Waiting antes que el hilo B, es decir $CS_A \rightarrow CS_B$. Por lo tanto, el algoritmo de Peterson para 2 hilos cumple con la propiedad de Justicia (FIFO).

5. **[30 min/20 puntos]** Peterson es una implementación de un candado para dos hilos. En la implementación siguiente (*DoublePeterson*) se utilizan tres candados Peterson para realizar una implementación de un candado para 4 hilos; se asume que a cada hilo le corresponde un ID único entre 1 y 4.

   Recuerda que en el candado de Peterson se cumplen las propiedades de Exclusión mutua, Deadlock-free y Starvation-free. Argumenta si *DoublePeterson* cumple con:

   a) Exclusión mutua.

   b) Deadlock-free.

   c) Starvation-free.

   d) Justicia.

   Hint: Si cumple con la propiedad, muestra la invariante que se mantiene (en el caso de la Exclusión mutua y Justicia), o muestra la cosa buena que pasará (en el caso de Deadlock-free y Starvation-free). No hace falta que demuestres Peterson.

   Si no cumple con alguna propiedad, muestra una ejecución que lo justifique.

```
class DoublePeterson implements Lock { // Candado para n = 4 hilos
    Shared Variables:
        1: Lock lock1, lock2, lock3;
        2: DoublePeterson() {
        3:     lock1 = petersonLock();
        4:     lock2 = petersonLock();
        5:     lock3 = petersonLock();
        6: }

        7: void lock() {
        8:     int me = ThreadID.get(); // Cada hilo tiene un ID nico de 1 a 4
        9:     if (me == 1 or me == 2) {
        10:        lock1.lock(); // Los hilos 1 y 2 toman el candado 1
        11:    }
        12:    if (me == 3 or me == 4) {
        13:        lock2.lock(); // Los hilos 3 y 4 toman el candado 2
        14:    }
        15:    lock3.lock(); // Todos pueden tomar el candado 3
        16: }
```

```
        17: void unlock() {
        18:     int me = ThreadID.get();
        19:     lock3.unlock();
        20:     if (me == 1 or me == 2) {
        21:        lock1.unlock();
        22:    }
        23:    if (me == 3 or me == 4) {
        24:        lock2.unlock();
        25:    }
        26: }
}
```

Figura 2: Implementación de un candado "DoublePeterson".

a) Exclusión Mutua
La invariante sería, que para que cualquier hilo logre entrar a la sección crítica, debe haber ejecutado exitosamente `lock3.lock()`. Como `lock3` es un candado de Peterson, su invariante garantiza que a lo más, un solo hilo puede poseerlo a la vez. Al ser `lock3` el último paso del método `lock()` del `DoublePeterson`, es imposible que dos hilos esten en la CS simultaneamente. Por lo tanto, si cumple con Exclusión Mutua.

b) Deadlock-free
Si varios hilos intentan entrar, el sistema siempre avanzará. Sabemos que los candados de Peterson son deadlock-free. Si los hilos 1 y 2 compiten en `lock1`, al menos uno ganará. Si los hilos 3 y 4 compiten en `lock2`, al menos uno ganará. A lo más, habrá dos ganadores compitiendo por `lock3`. Como `lock3` también es deadlock-free, obligatoriamente se resolverá el empate y dejará pasar a uno. El sistema nunca se congela. Por lo tanto `DoublePeterson` es deadlock-free.

c) Starvation-free
Si un hilo particular (digamos el Hilo 1) quiere entrar a la CS, eventualmente lo logrará. Primero competirá por `lock1`, y como `Peterson` es starvation-free, el hilo 2 no puede permanecer ahí infinitamente, así que el hilo 1 eventualmente ganará `lock1`. Una vez con `lock1`, el hilo 1 competirá por `lock3`, en la que nuevamente este tiene la propiedad de starvation-free, por lo que su entrada eventual a la CS está garnatizada. Por lo tanto, `DoublePeterson` cumple starvation-free.

d) FIFO (Justicia)
Supongamos que se inició el doorway en todos lo hilos del sistema.

1) El hilo 1 quiere entrar. Ejecuta `lock1.lock()`. Realiza su doorway completo (levanta su bandera y se hace víctima en el candado 1). En ese momento, el sistema lo pausa, justo antes de evaluar su `while`, es decir $\texttt{D}_1 yatermin$.

1) `El hilo 3 quiere entrar. Ejecuta lock2.lock(). El hilo 3 pasa su doorway, y no se detiene en el ciclo while, ganando lock2.`
2) `El hilo 3 avanza y ejecuta lock3.lock(). Como nadie más está compitiendo al final todavía, pasa el doorway y no se detiene en el while y entra a la sección crítica.`

Entonces, el hilo 1 terminó su doorway global antes de que el hilo 3 empezara a perdir el candado ($D_1 \rightarrow D_3$), pero el hilo 3 logró entrar a la CS primero ($CS_3 \rightarrow CS_1$). Por lo tanto no cumple FIFO.